

1. Introduction

DiscoGAN is designed for **unpaired image-to-image translation**. Unlike traditional GANs that map noise to images, DiscoGAN learns to translate images from one domain to another without paired examples. For example:

- Domain A $\rightarrow$ domain B (e.g., cats $\rightarrow$ dogs)
- Domain B $\rightarrow$ domain A (dogs $\rightarrow$ cats)

The system includes two generators and two discriminators. Each generator learns to translate in one direction, while each discriminator evaluates realism in its domain. A reconstruction mechanism ensures the translated image preserves content structure.

2. Installing and Importing Dependencies

```
!pip install torch torchvision matplotlib tqdm
```

[Show hidden output](#)

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Subset
import matplotlib.pyplot as plt
import numpy as np
from tqdm import tqdm
import itertools
```

3. Device Setup

Colab provides GPU acceleration. Since GANs are computationally heavy, GPU significantly improves training speed.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device: ", device)
```

```
Using device:  cpu
```

4. Dataset Preparation (Creating Two Domains)

DiscoGAN requires **two separate but unpaired datasets**. Thus, we extract two classes and treat them as independent domains. No pairing is used—the model learns correspondences automatically.

We split CIFAR-10 into two domains:

- Domain A $\rightarrow$ Cats (label 3)
- Domain B $\rightarrow$ Dogs (label 5)

```
transform = transforms.Compose([
    transforms.Resize(32),
    transforms.ToTensor(),
    transforms.Normalize((0.5,),(0.5,))
])

dataset = torchvision.datasets.CIFAR10(
    root = './data',
    train = True,
    download = True,
    transform = transform
)
```

[Show hidden output](#)

```
cats = [i for i, (_, label) in enumerate(dataset) if label == 3]
dogs = [i for i, (_, label) in enumerate(dataset) if label == 5]

domain_A = Subset(dataset, cats)
domain_B = Subset(dataset, dogs)

loader_A = DataLoader(domain_A, batch_size=64, shuffle=True)
loader_B = DataLoader(domain_B, batch_size=64, shuffle=True)
```

5. Generator Architecture

The generator:

- compresses the image into a latent representation
- reconstructs it in the style of the target domain

DiscoGAN generators are typically encoder-decoder networks.

```
class Generator(nn.Module):
    def __init__(self):
        super().__init__()

        self.model = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1),
            nn.ReLU(),

            nn.Conv2d(64, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            nn.ConvTranspose2d(128, 64, 4, 2, 1),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            nn.ConvTranspose2d(64, 3, 4, 2, 1),
            nn.Tanh()
        )

    def forward(self, x):
        return self.model(x)
```

6. Discriminator Architecture

There are two discriminators A and B. Each discriminator:

- judges whether an image is real or generated
- operates only within its domain

```

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()

        self.model = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1),
            nn.LeakyReLU(0.2),

            nn.Conv2d(64, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2),

            nn.Flatten(),
            nn.Linear(128*8*8, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)

```

7. Initialising Models

```

G_AB = Generator().to(device)
G_BA = Generator().to(device)

D_A = Discriminator().to(device)
D_B = Discriminator().to(device)

```

8. Loss Functions and Optimisers

DiscoGAN uses two types of learning signals:

1. **Adversarial learning:** Ensures translated images look realistic.
2. **Reconstruction consistency:** Ensures the translated image can be mapped back to the original domain.

This prevents arbitrary translation and encourages content preservation.

```

adversarial_loss = nn.BCELoss()
reconstruction_loss = nn.L1Loss()

optimiser_G = optim.Adam(
    itertools.chain(G_AB.parameters(), G_BA.parameters()),
    lr = 0.0002
)

optimiser_D_A = optim.Adam(D_A.parameters(), lr=0.0002)
optimiser_D_B = optim.Adam(D_B.parameters(), lr=0.0002)

```

9. Training Loop

Training alternates between:

- Updating generators (to fool discriminators and reconstruct images)
- Updating discriminators (to detect fake images)

The reconstruction step enforces consistency: if $A \rightarrow B \rightarrow A$, we should get back the original image.

```

epochs = 10

for epoch in range(epochs):

    loop = tqdm(
        zip(loader_A, loader_B),
        total = min(len(loader_A), len(loader_B))
    )

```

```
for (real_A, _), (real_B, _) in loop:
```

```
    real_A = real_A.to(device)
    real_B = real_B.to(device)

    valid = torch.ones(real_A.size(0), 1).to(device)
    fake = torch.zeros(real_A.size(0), 1).to(device)

    # -----
    # Training Generators
    # -----
    optimiser_G.zero_grad()

    fake_B = G_AB(real_A)
    fake_A = G_BA(real_B)

    adv_loss_AB = adversarial_loss(D_B(fake_B), valid)
    adv_loss_BA = adversarial_loss(D_A(fake_A), valid)

    rec_A = G_BA(fake_B)
    rec_B = G_AB(fake_A)

    rec_loss = reconstruction_loss(rec_A, real_A) + \
               reconstruction_loss(rec_B, real_B)

    G_loss = adv_loss_AB + adv_loss_BA + rec_loss

    G_loss.backward()
    optimiser_G.step()

    # -----
    # Training Discriminator A
    # -----
    optimiser_D_A.zero_grad()
    real_loss_A = adversarial_loss(D_A(real_A), valid)
    fake_loss_A = adversarial_loss(D_A(fake_A.detach()), fake)
    D_A_loss = (real_loss_A + fake_loss_A)
    D_A_loss.backward()
    optimiser_D_A.step()

    # -----
    # Training Discriminator B
    # -----
    optimiser_D_B.zero_grad()
    real_loss_B = adversarial_loss(D_B(real_B), valid)
    fake_loss_B = adversarial_loss(D_B(fake_B.detach()), fake)
    D_B_loss = (real_loss_B + fake_loss_B)
    D_B_loss.backward()
    optimiser_D_B.step()

    print(f"Epoch [{epoch+1}/{epochs}]")
```

```
100%|██████████| 79/79 [01:37<00:00, 1.24s/it]
Epoch [1/10]
100%|██████████| 79/79 [01:34<00:00, 1.20s/it]
Epoch [2/10]
100%|██████████| 79/79 [01:33<00:00, 1.19s/it]
Epoch [3/10]
100%|██████████| 79/79 [01:34<00:00, 1.19s/it]
Epoch [4/10]
100%|██████████| 79/79 [01:34<00:00, 1.19s/it]
Epoch [5/10]
100%|██████████| 79/79 [01:33<00:00, 1.18s/it]
Epoch [6/10]
100%|██████████| 79/79 [01:34<00:00, 1.19s/it]
Epoch [7/10]
100%|██████████| 79/79 [01:34<00:00, 1.20s/it]
Epoch [8/10]
100%|██████████| 79/79 [01:33<00:00, 1.19s/it]
Epoch [9/10]
100%|██████████| 79/79 [01:33<00:00, 1.18s/it]Epoch [10/10]
```

10. Visualising Results

Qualitative evaluation being critical in GAN research, we visually compare the original image and the translated output.

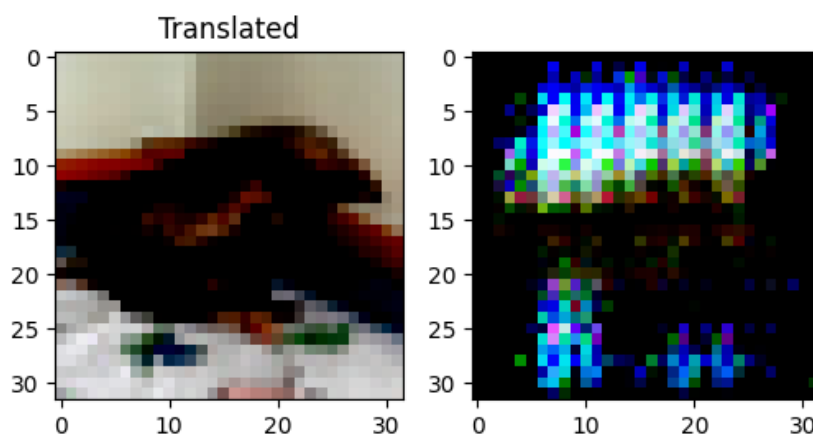
```
def show_images(real, fake):
    real = real.cpu().detach()
    fake = fake.cpu().detach()

    fig, axs = plt.subplots(1, 2, figsize=(6,3))
    axs[0].imshow(np.transpose(real[0], (1,2,0)))
    axs[0].set_title("Original")
    axs[1].imshow(np.transpose(fake[0], (1,2,0)))
    axs[1].set_title("Translated")
    plt.show()

real_sample, _ = next(iter(loader_A))
real_sample = real_sample.to(device)
fake_sample = G_AB(real_sample)

show_images(real_sample, fake_sample)
```

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o



11. Evaluation Strategy

Evaluation of DiscoGAN involves:

1. **Visual realism:** Does the translated image look like it belongs to the target domain?
2. **Content preservation:** Is the structure preserved?
3. **Diversity:** Does the model avoid producing identical outputs?
4. **Domain accuracy:** A pretrained classifier can measure whether translated images are classified as the target class.

Common metrics include FID score, classification accuracy, and human perceptual evaluation.